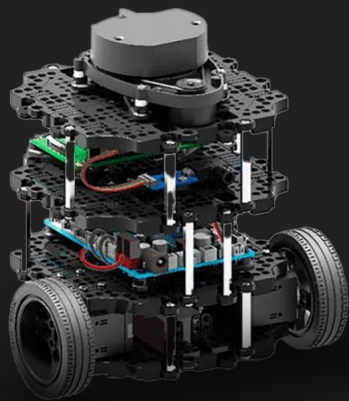
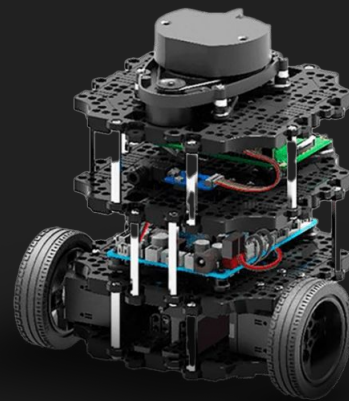




ROS2 TF

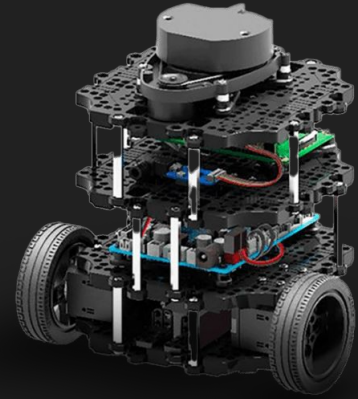
Course Project

Laith Sakkor



Project Overview:

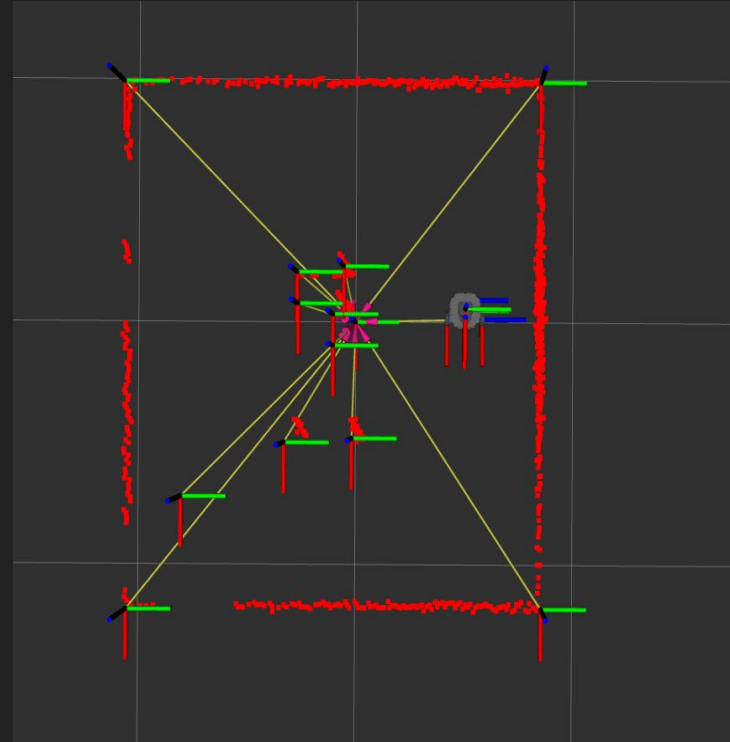
The objective was to implement TF-based coordinate tracking, detect environmental features, calculate simple trajectories, and execute navigation on a TurtleBot 3.



1. Simple Navigation :

create an “action_server” node that makes the robot follow the “waypoint_frame” :

1. Initialize Starting Point:
Manually drive the robot to a known starting position.
2. Track Waypoint:
Continuously calculate the robot's distance to the waypoint.
3. Use Euclidean Distance (2D):
To determine how far the robot is from the waypoint
Distance = $\sqrt{dx^2 + dy^2}$



3. Control Movement:

Control the robot by adjusting the linear and angular velocity based on the waypoint's position using simple proportional controller for robot navigation.

```
double dx = tf_wp.transform.translation.x;
double dy = tf_wp.transform.translation.y;
double distance = sqrt(dx * dx + dy * dy);
feedback->current_distance = distance;
goal_handle->publish_feedback(feedback);
```

```
geometry_msgs::msg::Twist cmd_vel;
double linear_speed = 0.5 * distance;
double angular_speed = 2.0 * std::atan2(dy, dx);
linear_speed = std::min(linear_speed, 0.5);
angular_speed = std::clamp(angular_speed, -1.0, 1.0);
```

```
cmd_vel.linear.x = linear_speed;
cmd_vel.angular.z = angular_speed;
cmd_vel_publisher->publish(cmd_vel);
geometry_msgs::msg::Twist stop_cmd;
```

1. Feature Detection :

Create a server to receive requests to detect corners and obstacles, and save their positions in a yaml file :

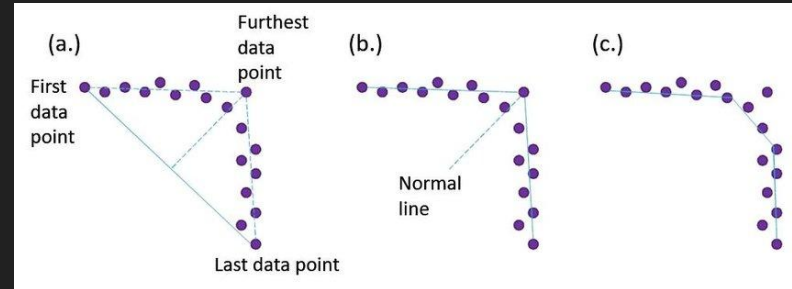
1. Corner And Obstacles Detection :

The first step was to select the point directly in front of the sensor, aligned with the front of the robot as the center point.

After that, depending on the request whether it's for detecting corners or obstacles the appropriate detection method is selected.

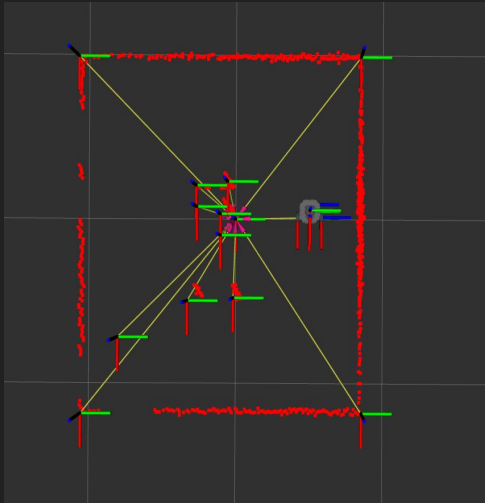
The method I used to detect corners is called the Split and Merge Line Segmentation algorithm. It is based on measuring the distance from a fitted line and is commonly used for edge or corner detection in 2D data.

```
auto scan = last_scan ;  
double angle_min = scan->angle_min;  
double angle_increment = scan->angle_increment;  
  
int center_idx = std::round((0.0 - angle_min) / angle_increment);  
int offset = std::round((M_PI / 4) / angle_increment);  
int start_idx = center_idx - offset;  
int end_idx = center_idx + offset;  
float angle=0;  
float range=0;  
RCLCPP_INFO(this->get_logger(), "start: %d", start_idx);  
RCLCPP_INFO(this->get_logger(), "end: %d", end_idx);
```



To detect obstacles, I consider any shape that the robot needs to move around as an obstacle. Based on this condition, a request is sent to detect and register it as an obstacle.

In corners and obstacles I got the position of the detected feature with respect to the laser scanner frame. Next, I got the position of this feature with respect to the odom frame using the TF Lookup technique.



```
geometry_msgs::msg::Point p;  
p.x = range * cos(angle);  
p.y = range * sin(angle);  
p.z = 0.0;  
  
    // Transform point to odom frame  
geometry_msgs::msg::PoseStamped scan_pose, odom_pose;  
scan_pose.header.frame_id = scan->header.frame_id;  
scan_pose.header.stamp = scan->header.stamp;  
scan_pose.pose.position = p;  
scan_pose.pose.orientation.w = 1.0;  
  
try {  
    odom_pose = tf_buffer_.transform(scan_pose, "odom");  
    best_point = odom_pose.pose.position;  
    found = true;  
  
    // Optional: Broadcast static TF for feature  
    geometry_msgs::msg::TransformStamped static_tf;  
    static_tf.header.stamp = now();  
    static_tf.header.frame_id = "odom";  
    static_tf.child_frame_id = request->feature + "_" + request->feature_id;  
  
    static_tf.transform.translation.x = best_point.x;  
    static_tf.transform.translation.y = best_point.y;  
    static_tf.transform.translation.z = best_point.z;  
    static_tf.transform.rotation.w = 1.0;  
  
    static_broadcaster_>sendTransform(static_tf);  
}
```

The final step is to save the positions of the detected corners and obstacles in a YAML file.

```
feature_frames:  
- feature: corner_1_frame  
  position:  
    x: 1.1408234420882342  
    y: 0.83661233826670445  
    z: 0.18865632904339708  
- feature: corner_2_frame  
  position:  
    x: 1.1449886639287197  
    y: -0.99040726909593668  
    z: 0.18844739891155834  
- feature: corner_3_frame  
  position:  
    x: 1.1449886639287197  
    y: -0.99040726909593668  
    z: 0.18844739891155834
```

1. Simple Trajectory :

Create a server to receive requests to that will broadcast one waypoint frame as Dynamic TF . This frame will be moved around the arena following predefined trajectory shape like a Rect with Rounded Corners

1. Read all the features from the static frames yaml file and broadcast all the features as Static TF

2. Calculate the x and y coordinates for the center point of the robot arena from the features coordinate .

```
YAML::Node root = YAML::LoadFile(
    "/home/latih/ros2-tf-real-robot-project/ros2_ws/src/ros2_tf_project
YAML::Node feature_frames = root["feature_frames"];

for (const auto &item : feature_frames)
{
    Feature f;
    f.name = item["feature"].as<std::string>();
    f.position.x = item["position"]["x"].as<double>();
    f.position.y = item["position"]["y"].as<double>();
    f.position.z = item["position"]["z"].as<double>();

    geometry_msgs::msg::TransformStamped static_tf;
    static_tf.header.stamp = rclcpp::Time(0);
    static_tf.header.frame_id = "odom";
    static_tf.child_frame_id = f.name;

    static_tf.transform.translation.x = f.position.x;
    static_tf.transform.translation.y = f.position.y;
    static_tf.transform.translation.z = f.position.z;
    static_tf.transform.rotation.w = 1.0;

    static_broadcaster->sendTransform(static_tf);
    rclcpp::sleep_for(std::chrono::milliseconds(10));
}
```


3. Calculate the x and y coordinates for the desired trajectory shape

4. Broadcast the position of waypoint frame with respect to the odom frame to move it around the arena as Dynamic TF

```
if (request->clockwise) {
    for (auto point : path) {
        geometry_msgs::msg::TransformStamped tf;
        tf.header.stamp = this->get_clock()->now(); // ✓ correc
        tf.header.frame_id = "center";
        tf.child_frame_id = "waypoint_frame";
        tf.transform.translation.x = point.x;
        tf.transform.translation.y = (double)(0.0);
        tf.transform.translation.z = 0.0;
        tf.transform.rotation.w = 1.0;
        tf_broadcaster->sendTransform(tf);
        rclcpp::sleep_for(500ms); // ~10Hz
    }
} else {
    for (auto it = path.rbegin(); it != path.rend(); ++it) {
        geometry_msgs::msg::TransformStamped tf;
        tf.header.stamp = this->get_clock()->now();
        tf.header.frame_id = "center";
        tf.child_frame_id = "waypoint_frame";
        tf.transform.translation.x = it->x;
        tf.transform.translation.y = it->y;
        tf.transform.translation.z = 0.0;
        tf.transform.rotation.w = 1.0;
        tf_broadcaster->sendTransform(tf);
        rclcpp::sleep_for(500ms);
    }
}
```

```
void trajectory()
{
    double w=1.8;
    double h=1.2;
    double r=0.2;
    int arc_points = 10;
    double z_map = 0.18891701572230546;

    double x_min = x_center - (w/2) +r;
    double x_max = x_center - 0.2 + (w/2) -r;
    double y_min = y_center - (h/2) +r;
    double y_max = y_center + 0.16 + (h/2) -r;
    geometry_msgs::msg::Point p;

    for (double x = x_max; x >= x_min; x -= 0.05) {
        p.x = x;
        p.y = y_min -r;
        p.z = z_map;
        path.push_back(p);
    }

    for (int i = 0; i <= arc_points; ++i) {
        double theta = M_PI * (1.5 - i / (double)arc_points * 0.5);
        p.x = x_min + r * cos(theta);
        p.y = y_min + r * sin(theta);
        p.z = z_map;
        path.push_back(p);
    }
}
```